Cysee — Arquitetura do Serviço de Monitoramento

Modelagem C4 + ADR, decomposição elemento a elemento, a partir de Cylock - Projeto e Plano de Pesquisa v1.2 (SNMP).docx

Cysee é um sistema distribuído de coleta, integridade e correlação de eventos de rede via SNMP e Syslog, desenvolvido pela Liga Acadêmica de Cibersegurança Cylock (UFS) como projeto de pesquisa. Este documento decompõe a arquitetura em camadas C4 (contexto, containers, componentes, fluxo dinâmico e rede), explicando cada elemento dos diagramas, e fecha com as decisões de arquitetura (ADR) que sustentam essas escolhas.

1. Contexto

Visão de mais alto nível: quem usa o Cysee e com quais sistemas externos ele troca dados. Nenhum detalhe interno aparece aqui.

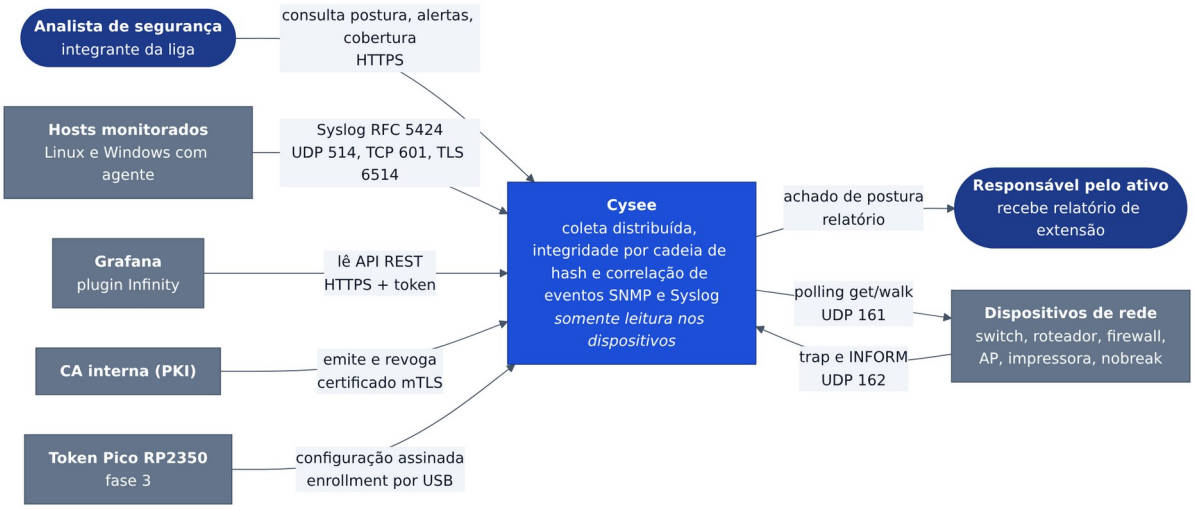


Figura 1: Contexto do Cysee (diagrama C4, nível 1).

- Analista de segurança — integrante da liga; consulta postura SNMP e alertas abertos via HTTPS.
- Responsável pelo ativo — recebe achado de postura SNMP como relatório/extensão (não acessa o sistema diretamente).
- Cysee — o sistema em si: coleta distribuída, integridade (hash chain) e correlação de eventos SNMP e Syslog.
- Dispositivos de rede — switch, roteador, firewall, AP, impressora; só falam SNMP (trap/INFORM na porta UDP 162, e respondem a polling get/walk na porta UDP 161). Cysee nunca escreve neles — é leitura apenas.
- Hosts monitorados — Linux/Windows com o agente Cysee instalado; emitem Syslog (UDP 514, TCP 601, ou TLS 6514).

- Grafana — ferramenta externa de dashboard; lê a API REST do núcleo central (somente leitura, com token) pelo plugin Infinity, sem acesso direto ao banco.
- CA interna — autoridade certificadora que emite e revoga o certificado de cada unidade (mTLS); revogar um certificado basta para parar a ingestão daquela unidade, sem passo manual extra no central.
- Token Pico RP2350 — dispositivo físico de enrollment (fase 3); apresenta configuração assinada para provisionar um host novo sem digitar comando.

2. Containers

Decompõe o Cysee nas duas unidades executáveis que de fato rodam: o agente (um processo por unidade monitorada) e o núcleo central (processo único, sem cluster).

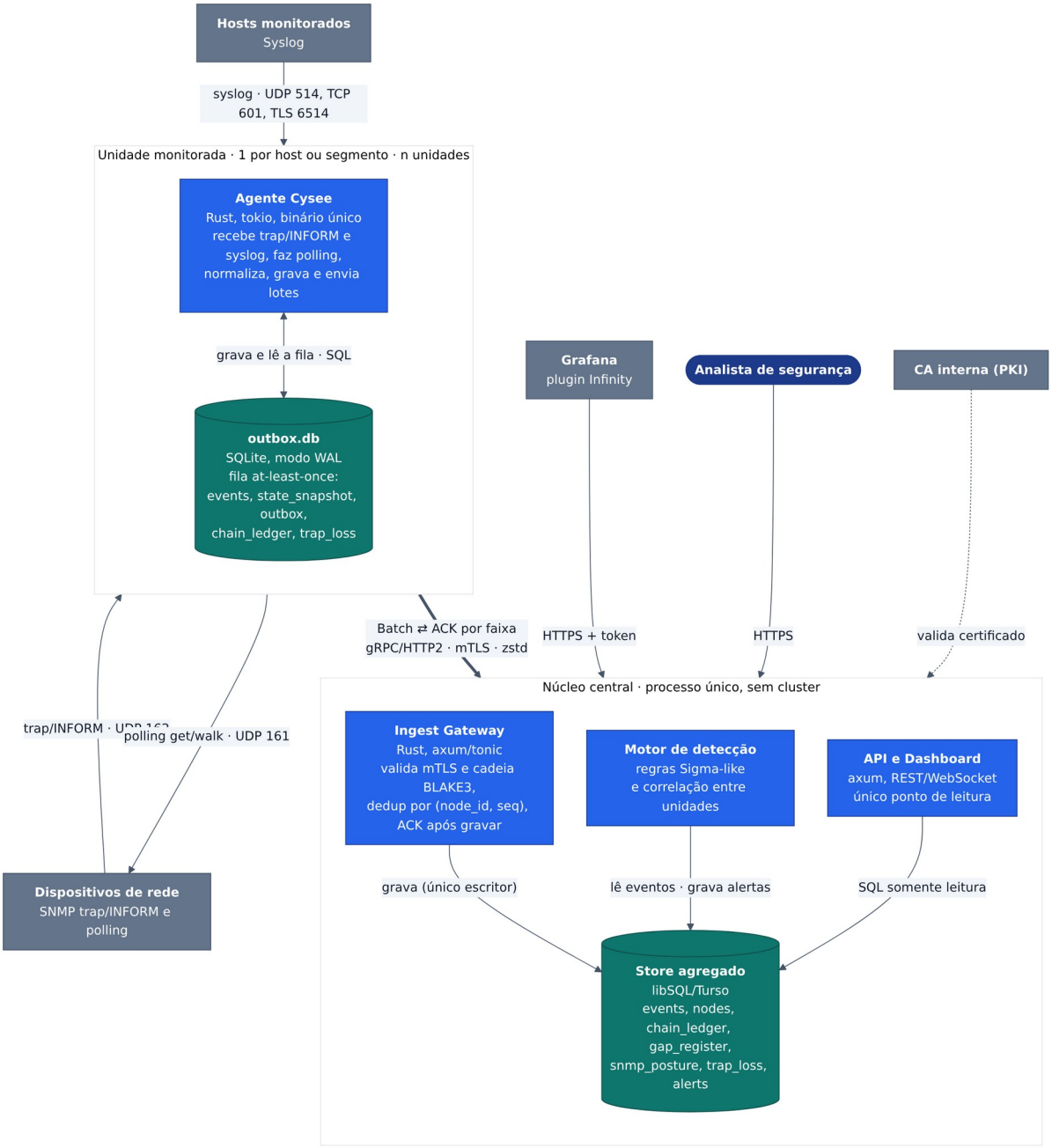


Figura 2: Containers: agente por unidade e núcleo central (C4, nível 2).

- Agente Cysee (Rust + tokio, binário único) — roda em cada unidade monitorada (1 por host/segmento, n unidades). Recebe trap/INFORM, syslog, faz polling de OIDs; normaliza; grava numa fila local; monta e envia lote ao central.
- outbox.db (SQLite modo WAL) — fila persistente at-least-once do agente: tabelas events, state_snapshot, outbox, chain_ledger. Garante que nenhum evento se perde mesmo se a rede cair.
- Ingest Gateway (Rust + axum, no central) — valida mTLS e a cadeia de hash BLAKE3 do lote recebido; faz dedup por (node_id, seq); só confirma (ACK) depois de gravar. É o único escritor do store central.
- Motor de detecção (Rust, no central) — avalia regras Sigma-like e correlação entre unidades sobre cada evento recém-gravado.
- API e Dashboard (Rust + axum, REST/WebSocket, no central) — expõe consulta de postura, cobertura e alertas; exporta CSV/Parquet. É o único ponto de leitura do store agregado, inclusive para o Grafana (plugin Infinity).
- Store agregado (libSQL/Turso, no central) — particionado por node_id + mês; tabelas events, nodes, snmp_posture, trap_loss, gap_register, alerts.
- Nota do diagrama: offline o agente acumula em outbox e nunca descarta (at-least-once + ACK por node_id/seq). O central não tem fila de mensagens (Kafka/RabbitMQ) porque o Ingest Gateway como único escritor já resolve idempotência — menos um sistema para a equipe manter.

3. Componentes do Agente

Abre o container Agente Cysee em seus módulos internos — o que roda dentro do processo Rust/tokio instalado em cada unidade.

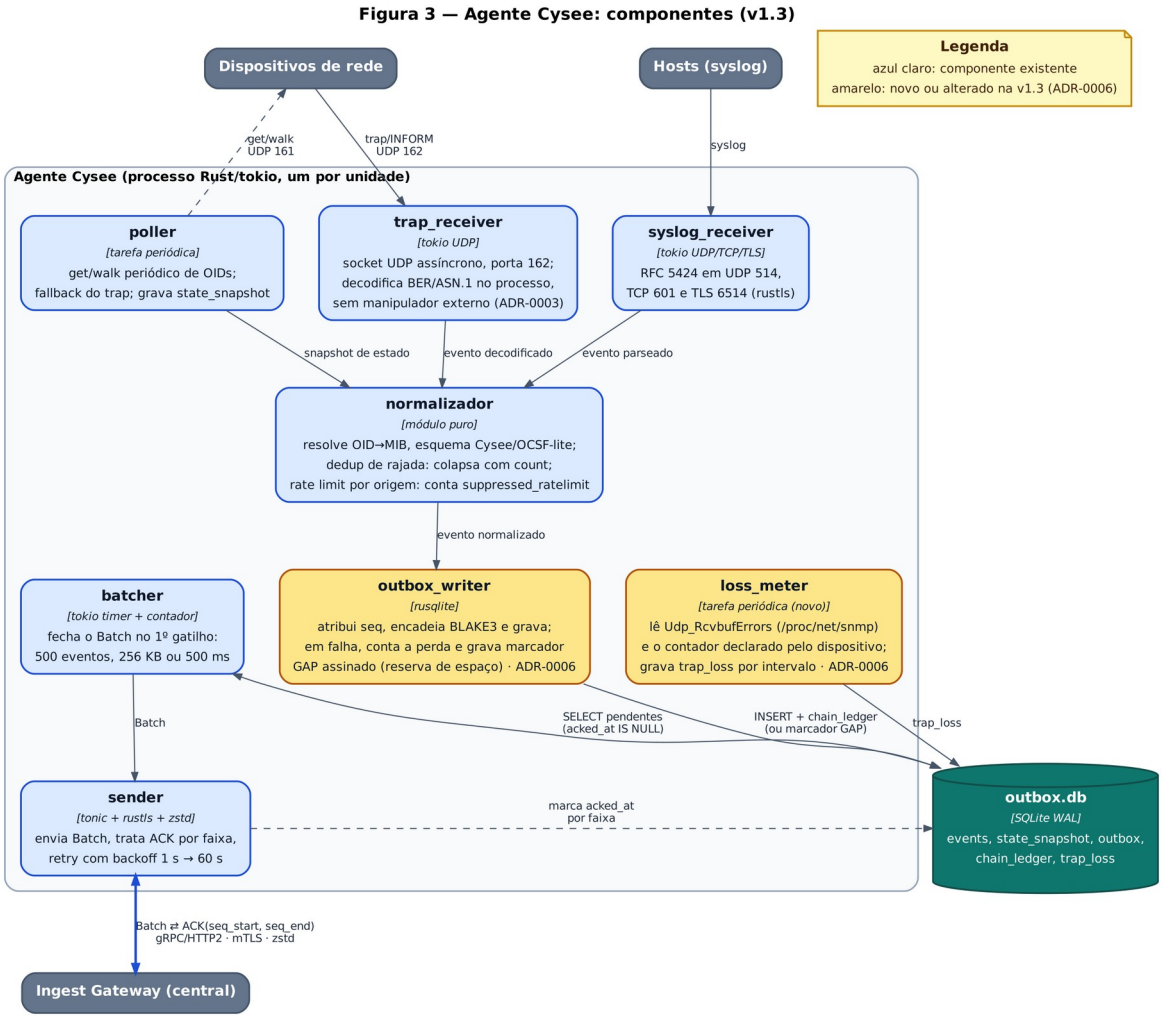


Figura 3: Componentes do agente (C4, nível 3).

- trap_receiver — socket UDP assíncrono (tokio) na porta 162; decodifica BER/ASN.1 sem delegar a um manipulador externo em série (ver ADR-0003).
- syslog_receiver — listener tokio UDP/TCP/TLS; recebe RFC 5424 em UDP 514, TCP 601, ou TLS 6514 via rustls.
- poller — tarefa periódica tokio; faz get/walk de OIDs como fallback do trap, grava state_snapshot.
- normalizador — módulo puro; resolve OID → MIB, aplica o esquema Cysee/OCSF-lite, faz dedup de rajada (colapsa eventos idênticos em um só, com count, first_ts e last_ts) e rate limit por origem (cada descarte incrementa suppressed_ratelimit), alimentando a equação de perda do ADR-0006.
- outbox_writer — usa rusqlite (SQLite WAL); grava o evento normalizado e atualiza o chain_ledger (cadeia BLAKE3 encadeada por node_id, nunca sobrescrita).
- batcher — tarefa tokio com timer + contador; monta o Batch quando bate o primeiro gatilho entre max_events=500, max_bytes=256KB ou max_wait=500ms.
- sender — usa tonic (cliente gRPC) + rustls + zstd; envia o Batch, trata o ACK por faixa,

faz retry com backoff exponencial de 1s a 60s.

- outbox.db — o mesmo banco SQLite WAL citado nos containers; aqui aparece como alvo de leitura/escrita de cada componente.

4. Componentes do Núcleo Central

Abre o container Núcleo central em seus módulos internos — processo único Rust que recebe, valida, grava, analisa e expõe os dados agregados.

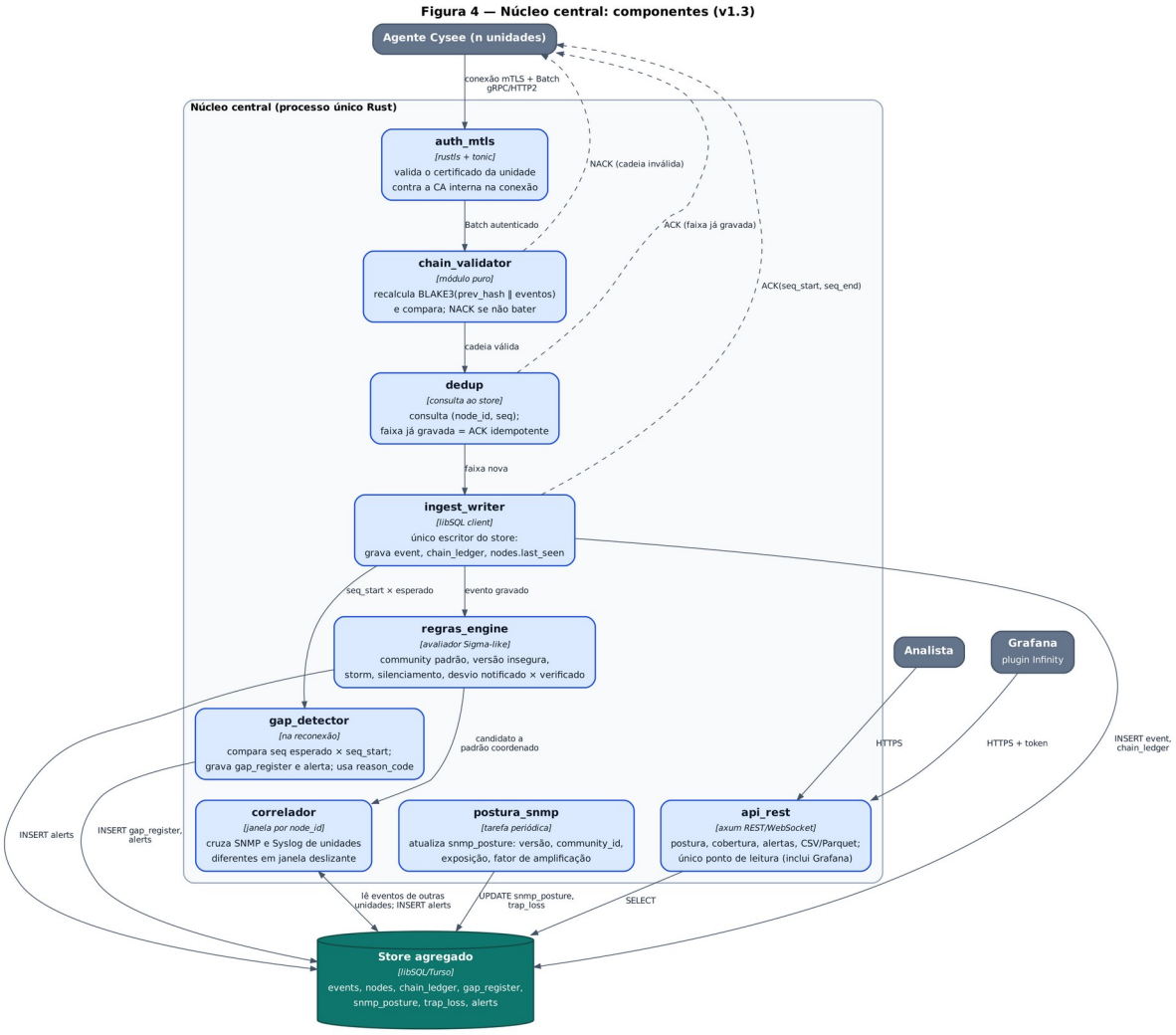


Figura 4: Componentes do núcleo central (C4, nível 3).

- auth_mtls — interceptor rustls + tonic; valida o certificado da unidade contra a CA interna na conexão.
- chain_validator — módulo puro; recalcula BLAKE3(prev_hash || events) e compara com o chain_hash recebido no lote — rejeita (NACK) se não bater.
- dedup — consulta o store por (node_id, seq); rejeita faixa já gravada sem erro, dando idempotência ao reenvio do agente.
- ingest_writer — único escritor do store (cliente rusqlite/libSQL); grava event, atualiza

chain_ledger e nodes.last_seen.

- gap_detector — roda na reconexão; compara o seq esperado (último confirmado) com o seq_start recebido; registra gap_register se houver lacuna.
- regras_engine — avaliador Sigma-like; avalia cada evento recém-gravado contra regras como community padrão, versão insegura, storm, desvio notificado × verificado.
- correlador — janela deslizante entre node_id diferentes; cruza SNMP e Syslog de unidades distintas, agrega padrão coordenado (ex.: storm de linkDown em várias unidades).
- postura_snmp — tarefa periódica; atualiza snmp_posture com versão, community_id, exposição e fator de amplificação do dispositivo.
- api_rest — axum REST/WebSocket; expõe consulta de postura, cobertura, alertas; exporta CSV/Parquet.
- Store agregado (libSQL/Turso) — único destino de escrita de todo o central; nota do diagrama reforça: sem fila de mensagens, o ingest_writer como único escritor evita precisar de Kafka/RabbitMQ.

5. Fluxos dinâmicos

Cada diagrama abaixo segue um fluxo de negócio ponta a ponta, numerando as interações entre componentes já apresentados nas seções 3 e 4.

5.1 Recepção e normalização de um evento

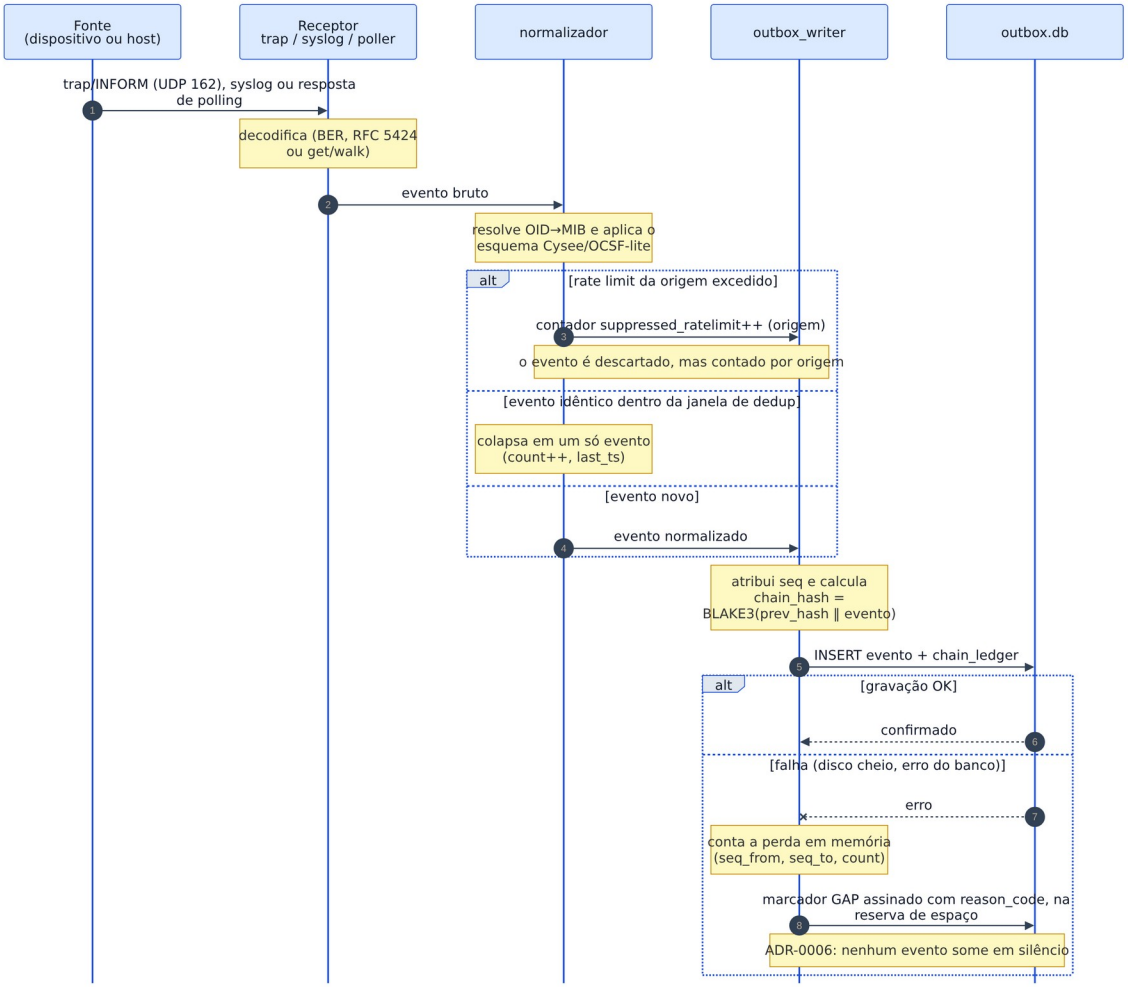


Figura 5: Agente: recepção, normalização e gravação de um evento, com a política de falha do ADR-0006.

trap_receiver, syslog_receiver e poller são três origens alternativas do mesmo tipo de evento — não passos sequenciais; por isso a numeração 1/2 se repete em cada origem. (1) cada receptor decodifica seu formato de origem (BER para trap, RFC 5424 para syslog, get/walk para poll); (2) envia o evento bruto ao normalizador. A partir daí o fluxo converge: (3) o normalizador resolve OID→MIB, aplica rate limit por origem e dedup por (OID, origem, varbinds, janela), que colapsa eventos idênticos em um só com count, first_ts e last_ts, enquanto cada descarte por rate limit incrementa suppressed_ratelimit; (4) envia o evento normalizado ao outbox_writer; (5) o outbox_writer atribui o seq, calcula chain_hash = BLAKE3(prev_hash || evento) e grava o evento, atualizando o chain_ledger. Nota crítica: se a gravação falhar (ex.: disco cheio), o agente conta a perda em memória e grava um marcador GAP assinado, com seq_from, seq_to, count e reason_code, assim que o disco responder, usando uma reserva de espaço exclusiva para marcadores. Nenhum evento é descartado em silêncio (ADR-0006). Observação sobre a figura 5: o rótulo da nota no diagrama ainda descreve o comportamento anterior (descarte com log) e deve ser atualizado ao reexportar a figura.

5.2 Flush do lote e ACK por faixa

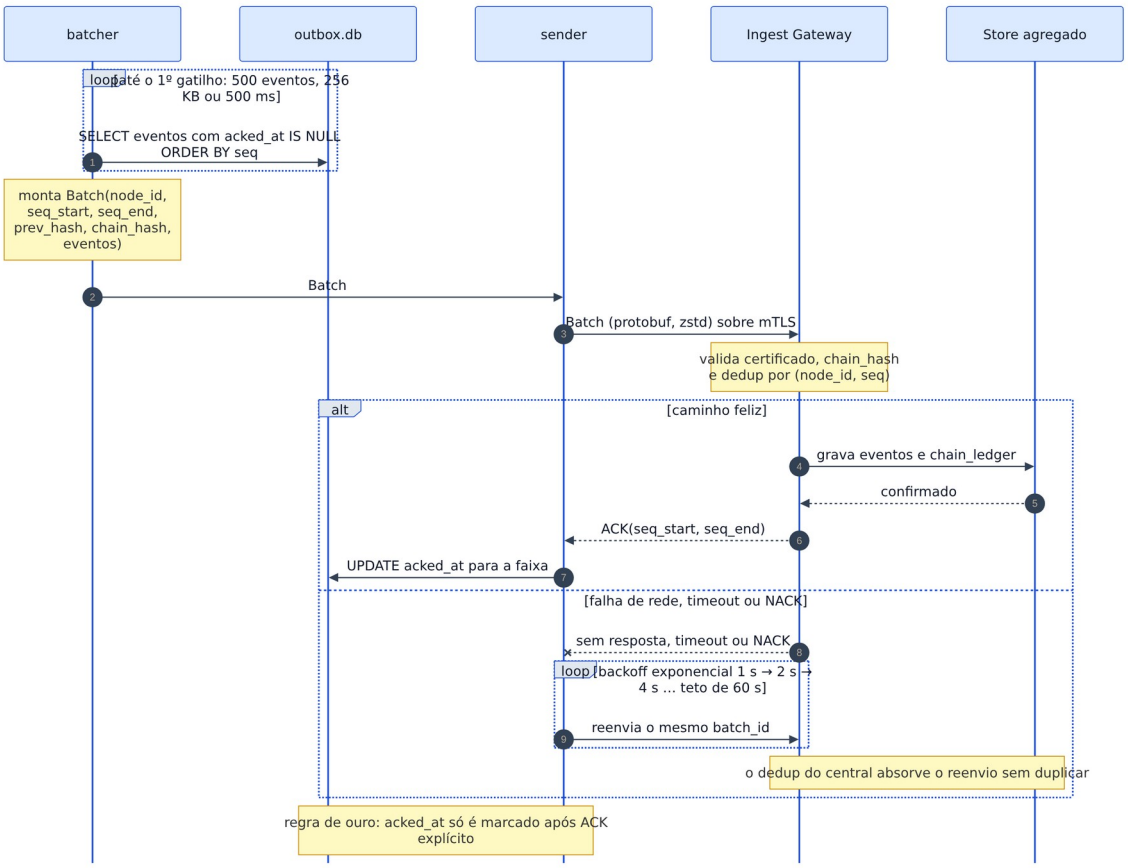


Figura 6: Agente: flush do lote e ACK por faixa (ADR-0005, max_wait de 500 ms).

(1) o batcher consulta o outbox por eventos ainda não confirmados (acked_at IS NULL), ordenados por seq; o gatilho de disparo é o primeiro entre max_events=500, max_bytes=256KB ou max_wait=500ms. (2) monta o Batch com node_id, seq_start, seq_end, prev_hash, chain_hash e a lista de eventos; (3) entrega ao sender; (4) o sender envia ao Ingest Gateway do central via protobuf, compactado em zstd, sobre mTLS. (5) o gateway valida o chain_hash e faz dedup por (node_id, seq). No caminho feliz, (6) o gateway responde ACK(seq_start, seq_end) e (7) o sender atualiza acked_at da faixa no outbox. Em falha de rede, timeout ou NACK, o sender reenvia o mesmo batch_id com backoff exponencial (1s → 2s → 4s...cap 60s); o dedup do central absorve o reenvio sem duplicar no store. Regra de ouro: o outbox só marca uma faixa como confirmada após ACK explícito — nunca antes.

5.3 Detecção por regras e correlação entre unidades

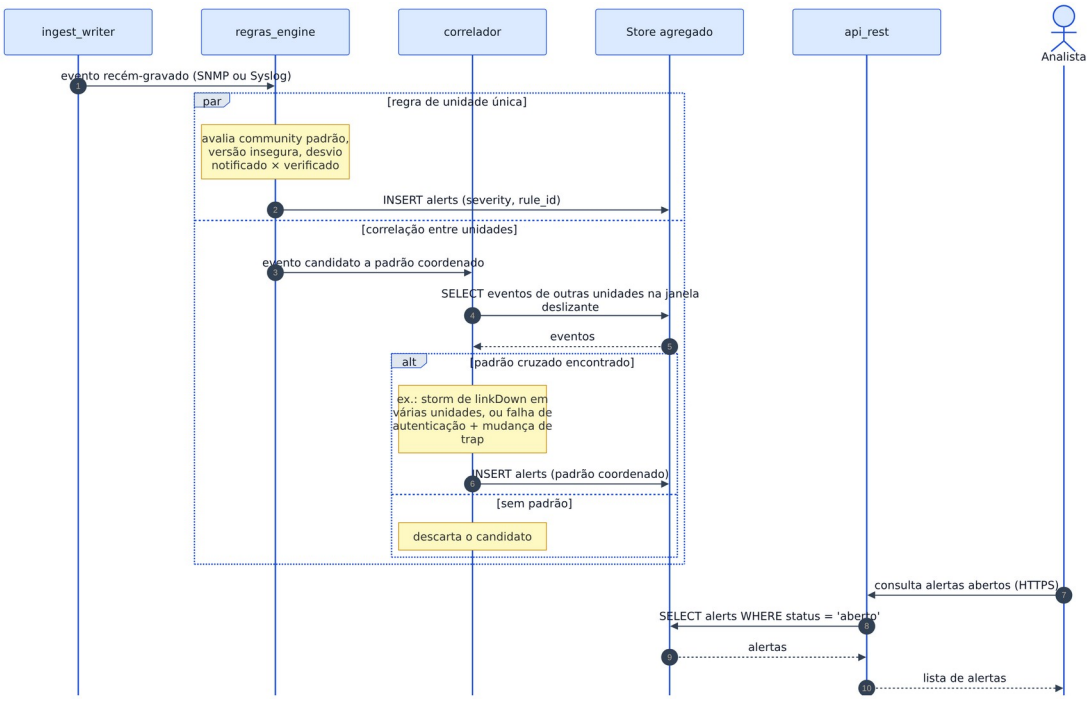


Figura 7: Central: detecção por regras e correlação entre unidades.

(1) o ingest_writer, logo após gravar, repassa o evento ao regras_engine. A partir daqui há dois caminhos independentes sobre o mesmo evento, não um encadeamento único: regra de unidade única — (2a) avalia community padrão, versão insegura, desvio notificado × verificado (trap vs poll); (3a) grava alerta (severity, rule_id) no store. correlação entre unidades — (2b) o evento é passado como candidato a padrão coordenado ao correlador; (3b) o correlador consulta eventos de outras unidades na mesma janela deslizante, buscando padrões como storm de linkDown em várias unidades ou falha de autenticação em uma unidade somada a mudança de trap em outra; sem padrão cruzado, descarta o candidato; (4b) se encontrar padrão, grava alerta. Por fim, (5) o analista consulta alertas abertos pela API REST, que (6) seleciona do store onde status='aberto'. O formato Sigma-like garante que qualquer integrante da liga consiga ler e revisar a regra (ver ADR-0004).

5.4 Reconciliação após partição de rede

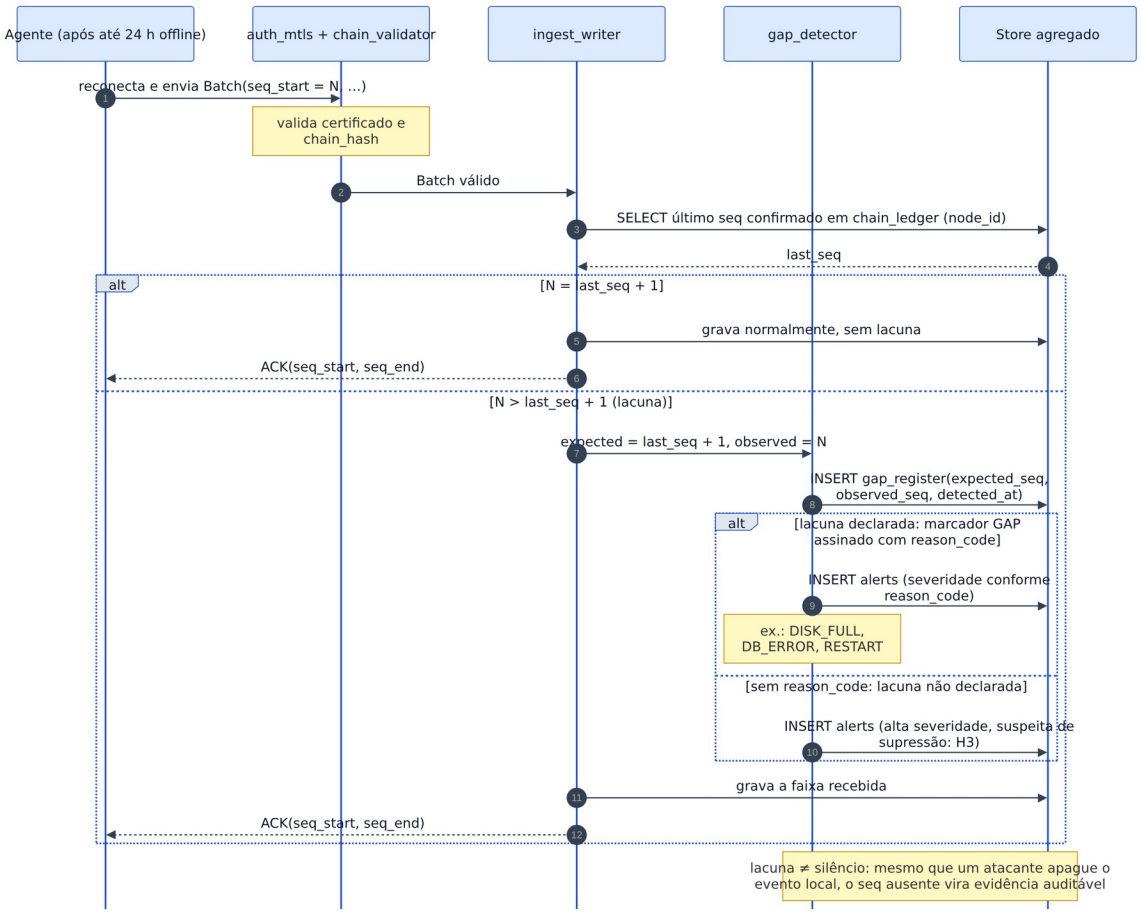


Figura 8: Central: reconciliação após partição de rede (ADR-0005 / H3).

(1) o agente reconecta depois de até 24h offline, enviando Batch(seq_start=N, ...); (2) o ingest_writer consulta no chain_ledger o último seq confirmado daquele node_id. Se seq_start == last_seq + 1, grava normalmente, sem lacuna. Se seq_start > last_seq + 1, há uma lacuna: (3) o gap_detector grava em gap_register (expected_seq, observed_seq, detected_at). A unidade pode anexar um reason_code assinado (ex.: disco cheio, reinício) no chain_ledger — sem reason_code, a lacuna é tratada como não declarada, portanto suspeita (hipótese H3 do projeto de pesquisa). (4) o gap_detector grava um alerta cuja severidade depende de haver ou não reason_code. A lição central: lacuna declarada não é igual a silêncio — mesmo quando um atacante apaga o evento local, a lacuna no seq fica registrada como evidência auditável.

5.5 Enrollment por token físico assinado (fase 3)

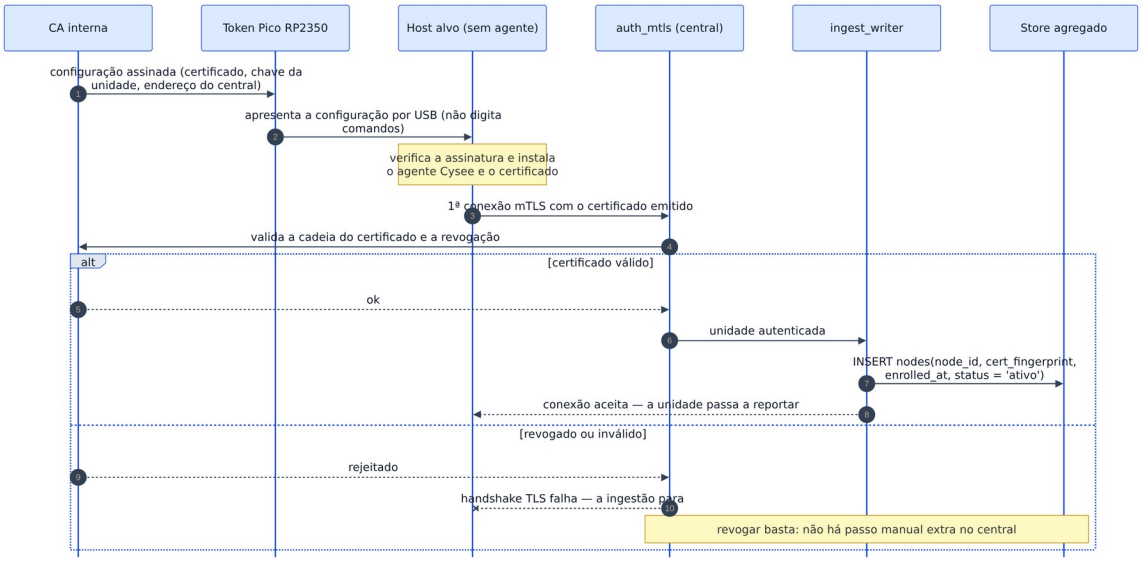


Figura 9: Provisionamento: enrollment por token físico assinado (fase 3).

(1) a CA interna entrega ao token Pico RP2350 uma configuração assinada contendo certificado e endereço do central; (2) o Pico porta essa configuração assinada ao host alvo — diferente de injeção HID, o Pico entrega configuração assinada, não comandos de teclado (que seriam frágeis e dependentes de layout/privilégio). (3) o host instala o agente Cysee e o certificado recebido; (4) faz a primeira conexão mTLS com o certificado emitido ao auth_mtls do central; (5) o auth_mtls valida a cadeia do certificado junto à CA. Se o certificado for válido, segue para (6) o ingest_writer grava a nova unidade em nodes (node_id, cert_fingerprint, enrolled_at, status='ativo') e (7) aceita a conexão — a unidade passa a reportar. Se o certificado estiver revogado ou inválido, a conexão é rejeitada (handshake TLS falha); revogar basta para parar a ingestão, sem passo manual extra no central.

6. Diagrama de rede

Visão física de implantação em laboratório (topologia EVE-NG, VLANs isoladas), mostrando onde cada container roda e como o isolamento de rede contém lateralmente cada unidade.

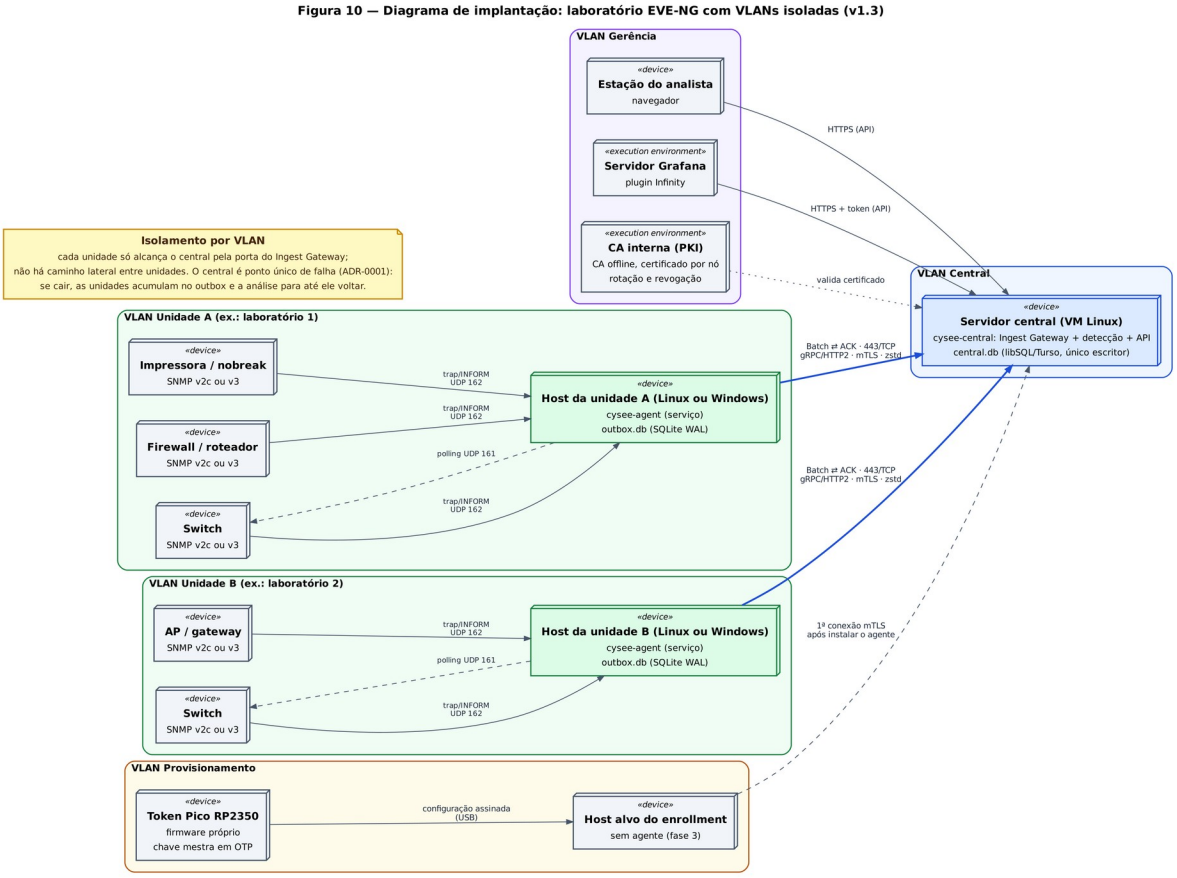


Figura 10: Implantação: laboratório EVE-NG com VLANs isoladas.

- VLAN Gerência — estação do analista, CA interna (PKI) e Grafana; alcança o central só pelas portas de consulta (HTTPS para a API, plugin Infinity do Grafana lendo a mesma API).
- VLAN Central — núcleo central (Ingest Gateway + detecção + API, um único processo Rust) e o Store agregado (libSQL/Turso); único escritor do store.
- VLAN Unidade A / VLAN Unidade B — cada uma representa um segmento monitorado real (ex.: um laboratório): host com o Agente Cysee, mais os dispositivos SNMP daquele segmento (switch, roteador, firewall, impressora, AP, nobreak). O agente recebe trap/INFORM (UDP 162) desses dispositivos e faz polling (UDP 161) neles.
- VLAN Provisionamento — token Pico RP2350 e o host alvo do enrollment (fase 3); o Pico porta a config assinada ao host por conexão física (USB), e o host só fala com o central depois de instalar o agente.
- Isolamento por VLAN: cada unidade só alcança o central pela porta do Ingest Gateway (gRPC/HTTP2 + mTLS + zstd) — nunca outra unidade; não há caminho lateral entre VLANs de unidades diferentes.
- Ponto único de falha aceito nesta edição (ADR-0001): o núcleo central não tem cluster nem alta disponibilidade — se cair, as unidades continuam coletando (acumulam em outbox local) mas a análise para até ele voltar.

7. Decisões de arquitetura (ADR)

As seis decisões abaixo formalizam as escolhas estruturais do projeto, com contexto, consequências aceitas e alternativas descartadas.

ADR-0001 — Agente único por unidade, núcleo central único sem cluster

Contexto: equipe de 10 estudantes, 10–15h/semana, sem orçamento de nuvem ou cluster; infraestrutura só de laboratório. Decisão: um binário Rust único por unidade, fazendo trap+syslog+polling no mesmo processo assíncrono; núcleo central único, sem cluster, sem fila de mensagens — o Ingest Gateway é o único escritor do store. Consequência aceita: o central é ponto único de falha (coleta continua, análise para até ele voltar); nenhuma mitigação nesta edição, réplica de leitura e recuperação automática ficam como trabalho futuro. Alternativas descartadas: cluster com quorum distribuído (fora do orçamento) e fila de mensagens entre agente e central (dedup por node_id/seq já resolve idempotência sem precisar de mais um sistema).

ADR-0002 — SQLite autônomo na borda (não réplica embutida do Turso)

Contexto: o modo de réplica embutida do libSQL/Turso encaminha a escrita ao primário remoto — sem rede, a escrita para, o oposto do que o agente precisa (acumular offline). Decisão: o banco da unidade é SQLite puro em modo WAL, autônomo, sem sincronização automática de esquema com o central; libSQL/Turso fica só no núcleo central; a reconciliação é o protocolo próprio de lote e faixa de seq (ADR-0005). Consequência aceita: perde-se sincronização automática de esquema — qualquer migração precisa de versionamento manual. Alternativas descartadas: réplica embutida do Turso (escrita offline não funciona) e RocksDB na unidade (menos prático para reconciliação por faixa).

ADR-0003 — Recepção de trap sem manipulador externo em série

Contexto: o padrão do setor (snmptrapd) entrega trap a manipuladores externos em série — um manipulador de 200ms limita a cinco traps por segundo, e o buffer transborda; gargalo conhecido que o projeto mede e evita. Decisão: receber, decodificar (BER/ASN.1) e gravar em disco dentro do próprio processo tokio, sem manipulador externo no caminho crítico; rate limiting e dedup de rajada ficam no normalizador, não em processo separado. Consequência aceita: decodificação de BER e tratamento de pacote malformado passam a ser responsabilidade do próprio código (não delega ao Net-SNMP), inclusive validação contra pacote malicioso — ponto que nunca deve ser relaxado, é requisito de não perder evento em rajada. Alternativa descartada: Net-SNMP via chamada externa (reintroduziria o próprio gargalo de serialização que o projeto evita).

ADR-0004 — Detecção por regras (Sigma-like), não aprendizado de máquina

Contexto: equipe de 10 estudantes em tempo parcial, 12 semanas; regras precisam ser auditáveis por qualquer integrante da liga. Decisão: motor de detecção em formato próximo do Sigma — community padrão, versão insegura, storm coordenado, silenciamento de ativo, desvio notificado × verificado; correlação entre unidades roda sobre o mesmo motor, janela deslizante por node_id. Consequência aceita: cobertura menor de anomalia desconhecida comparado a ML; trabalho futuro

prevê testar um modelo simples como comparação (não substituição), quando houver conjunto rotulado. Alternativas descartadas: ML em produção (fora do escopo e do tempo da equipe nesta edição) e regras ad-hoc em código (menos portável e mais difícil de revisar que o formato Sigma-like).

ADR-0005 — Formato e gatilho de flush do lote

Contexto: o esquema já definia a tabela outbox e o transporte (gRPC/HTTP2 + mTLS + zstd), mas faltava o gatilho de quando fechar e enviar um lote, e o formato exato da mensagem. Decisão: gatilho de flush é o primeiro entre max_events=500, max_bytes=256KB (antes do zstd) ou max_wait=500ms desde o primeiro evento pendente. Mensagem protobuf Batch{node_id, seq_start, seq_end, prev_hash, chain_hash, events[]}, com chain_hash = BLAKE3(prev_hash || bytes serializados dos eventos em ordem de seq). ACK por faixa (seq_start..seq_end); o agente só marca acked_at depois do ACK explícito. Retry com backoff exponencial 1s→2s→4s...cap 60s, reenviando o mesmo batch_id; dedup central por (node_id, seq) absorve o reenvio sem duplicar. Consequência aceita: os três parâmetros (max_events, max_bytes, max_wait) são ponto de partida, ainda não calibrados por medição real de rajada. O max_wait foi reduzido de 2 s para 500 ms porque 2 s consumiria todo o orçamento de latência p50 do projeto (abaixo de 2 s); a calibração será uma varredura de max_wait em {100, 500, 1000 ms} no benchmark da fase 2, e o resultado será registrado neste ADR. Alternativas descartadas: campo map no protobuf Event para varbinds (sem ordem determinística de serialização, quebraria o hash — usar repeated k/v) e flush só por tempo sem limite de tamanho (rajada de trap poderia gerar lote impraticável de enviar em 500ms).

ADR-0006 — Falha de gravação e contabilidade de perda no agente

Contexto: a versão anterior descrevia o descarte com log quando a gravação falhava, o que contradiz a hipótese H1 (perda zero) e a tese central do projeto, em que supressão vira evidência e não silêncio; além disso, a dedup e o rate limit do normalizador ocorrem antes do hash e alteram a contagem de eventos, o que distorce a medição de perda por origem (declarado × registrado). Decisão: o seq é atribuído ao evento antes da gravação; se a gravação falhar, o agente conta a perda em memória e, assim que o disco responder, grava no chain_ledger um marcador assinado GAP(seq_from, seq_to, count, reason_code), com códigos iniciais DISK_FULL, DB_ERROR e RESTART; uma reserva de espaço pré-alocada, exclusiva para marcadores, garante que disco cheio não impeça o registro da lacuna. A dedup nunca apaga: colapsa eventos idênticos em um só, com count, first_ts e last_ts, e o hash cobre o evento colapsado; o rate limit descarta, mas cada descarte incrementa suppressed_ratelimit por origem. A perda por origem é contabilizada pela equação declarado = registrado + dedup + rate limit + kernel + residual, e o termo residual é a perda desconhecida que o artigo reporta; a tabela trap_loss ganha as colunas suppressed_dedup e suppressed_ratelimit. Consequência aceita: um evento perdido antes da gravação em disco continua perdido, mas deixa de ser invisível, pois aparece como GAP ou como residual; H1 passa a valer após a gravação em disco; se o disco e a reserva se esgotarem, o contador em memória é gravado no próximo boot com reason_code RESTART. Alternativas descartadas: descartar com log local (o log é apagável pelo mesmo atacante e não chega ao central) e bloquear o recebimento até haver espaço (reintroduz o transbordamento do buffer do kernel, que o projeto mede e quer evitar).

8. Diagramas UML de apoio

Esta seção complementa o modelo C4 com diagramas UML que ajudam a entender, implantar e implementar o Cysee. Os diagramas de casos de uso, de classes, de entidade-relacionamento e de estados descrevem o sistema; os de implantação e de atividades orientam a instalação e o código.

8.1 Casos de uso

Mostra quem usa o Cysee e o que cada ator pode fazer: o analista consulta eventos, alertas e a integridade; o administrador ativa unidades e gerencia a CA; os equipamentos monitorados enviam traps e syslog. Serve de base para a lista de requisitos e para os testes de aceitação.

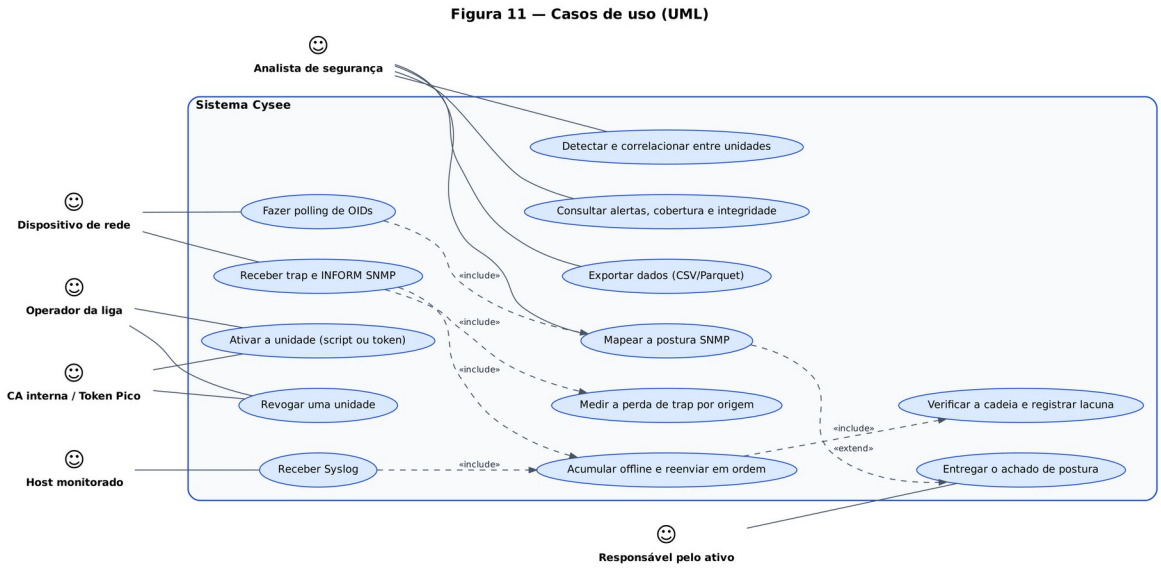


Figura 11: Casos de uso: atores e funções do Cysee.

8.2 Classes

O primeiro diagrama fixa o vocabulário do domínio (evento, lote, GAP, ledger) e o segundo mostra a organização do código em Rust: os traits que separam recepção, normalização, gravação e transporte, e as structs que os implementam. É o ponto de partida para criar os módulos e os testes unitários.

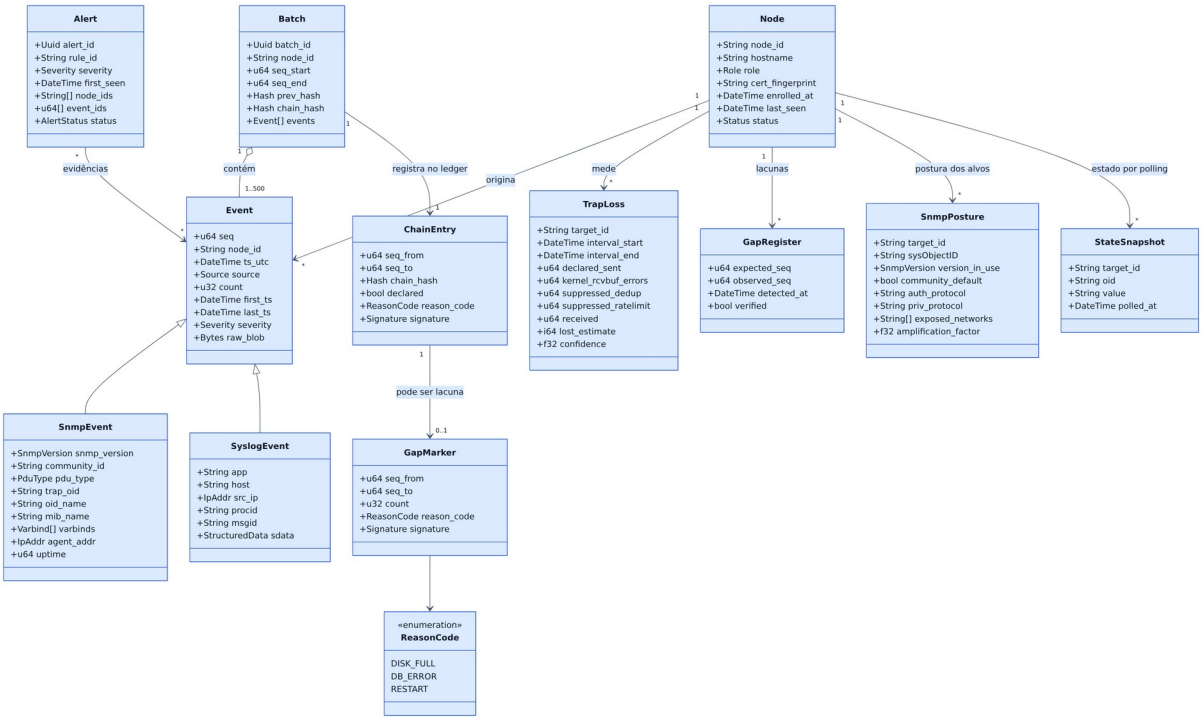


Figura 12: Classes do domínio: evento, lote, marcador GAP e contabilidade de perda.

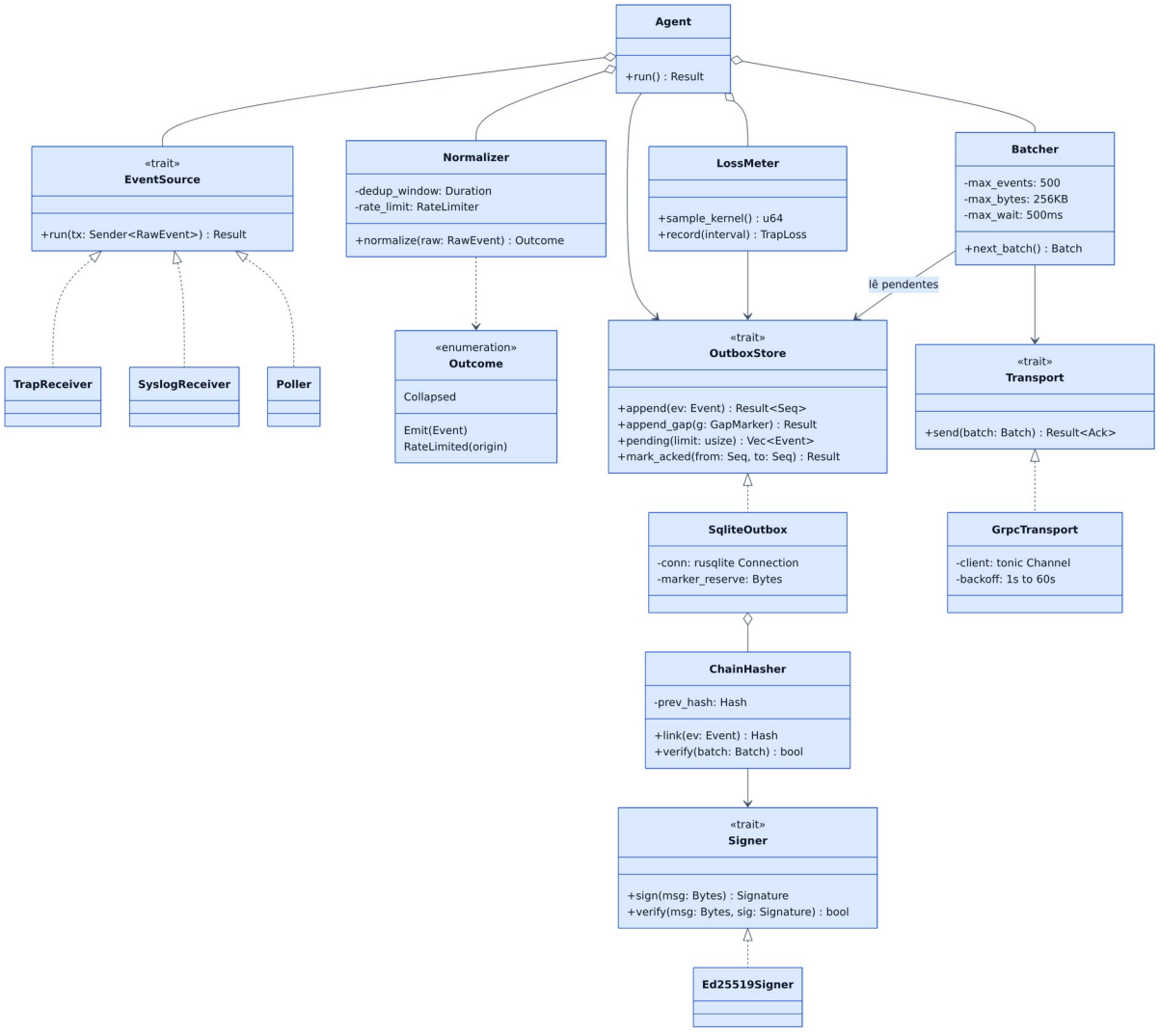


Figura 13: Classes e traits Rust do agente e do núcleo central.

8.3 Modelo de dados

Relaciona as tabelas do outbox.db (agente) e do central.db (núcleo), incluindo chain_ledger, nodes e trap_loss com as colunas suppressed_dedup e suppressed_ratelimit definidas no ADR-0006.

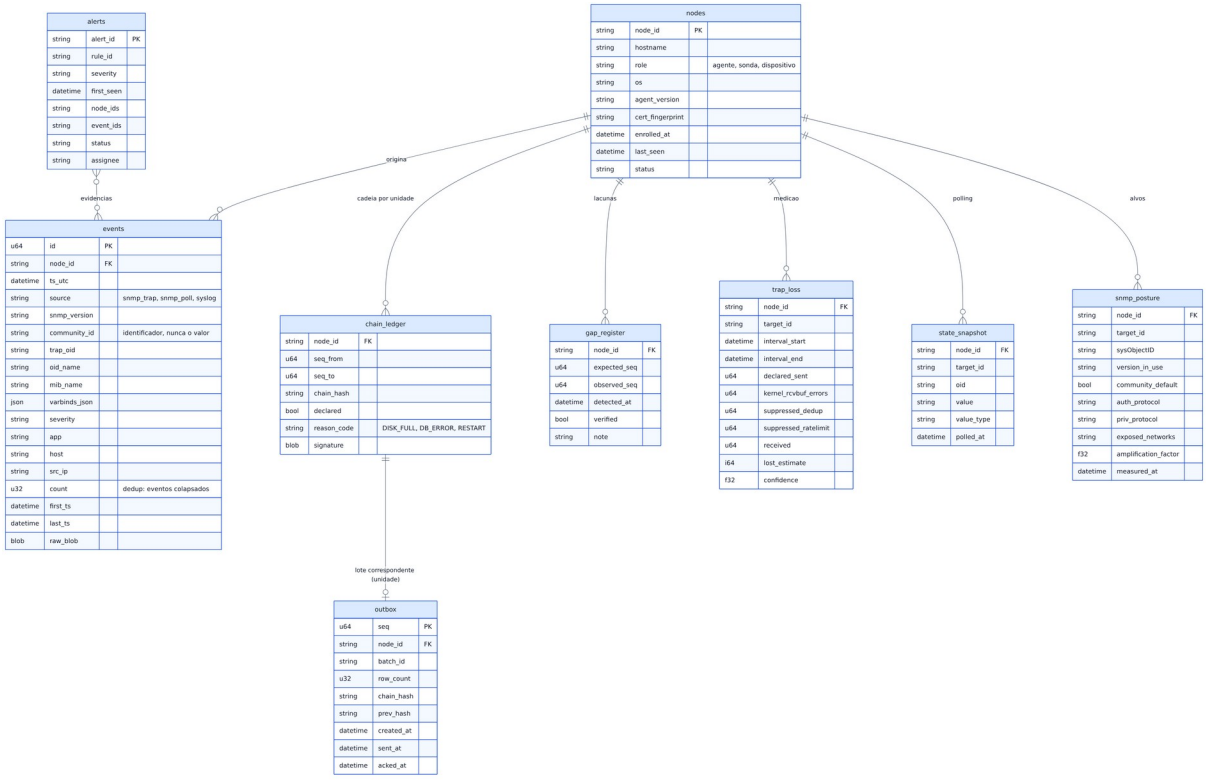


Figura 14: Modelo entidade-relacionamento do outbox da unidade e do central.db.

8.4 Máquinas de estados

O ciclo de vida do evento (recebido, gravado como PENDENTE, enviado, confirmado por faixa) e o do agente (normal, desconectado, disco cheio, recuperação) deixam explícito o que acontece em falha de rede ou de disco, e quando a lacuna vira GAP.

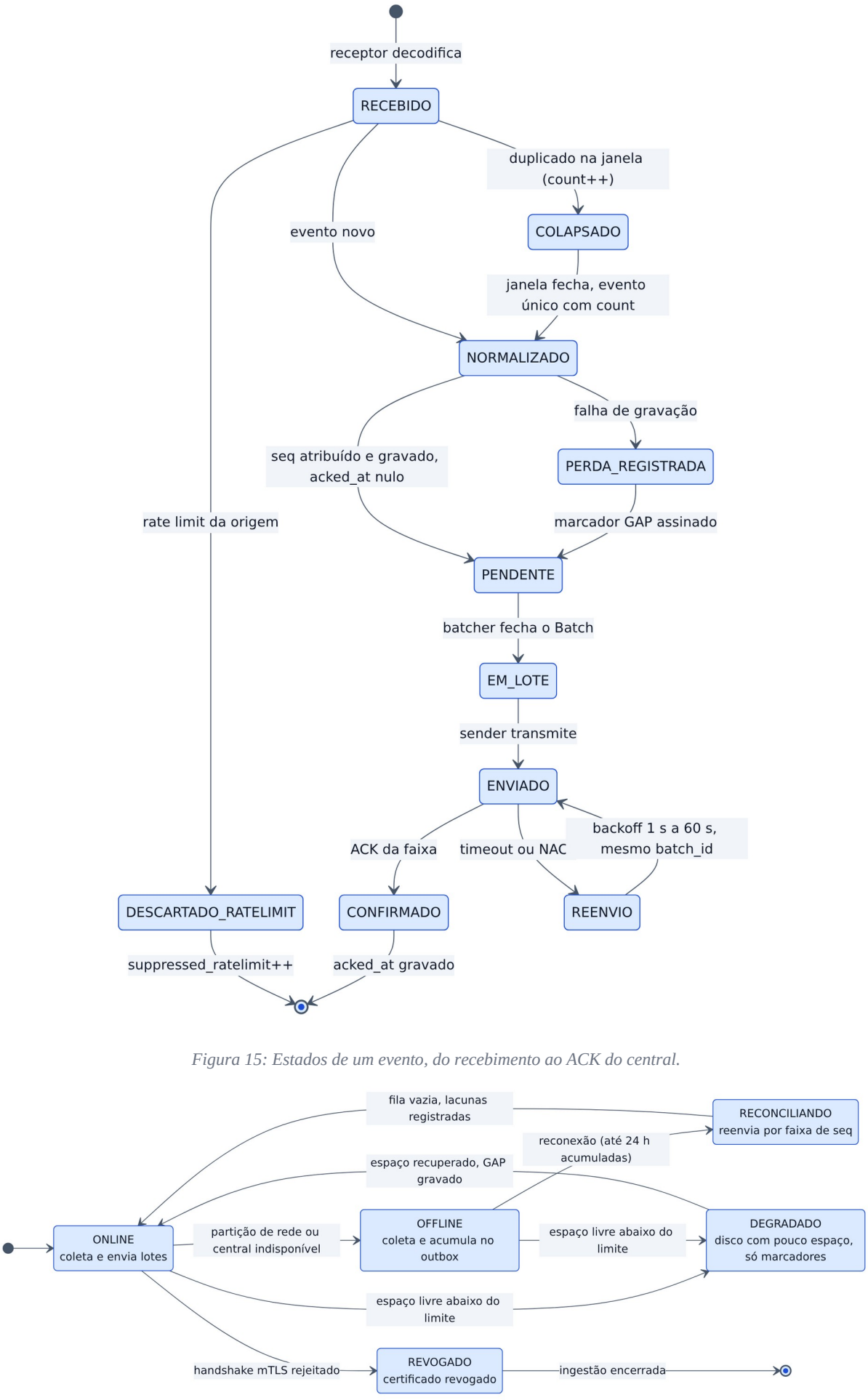


Figura 15: Estados de um evento, do recebimento ao ACK do central.

Figura 16: Estados do agente: operação normal, degradado e recuperação.

8.5 Atividade: política de falha de gravação

Detalha o ADR-0006 passo a passo: rate limit e dedup antes da gravação, cálculo do hash encadeado, INSERT no outbox e, em caso de falha, marcador GAP assinado na reserva de espaço, com gravação adiada para o próximo boot se a reserva se esgotar.

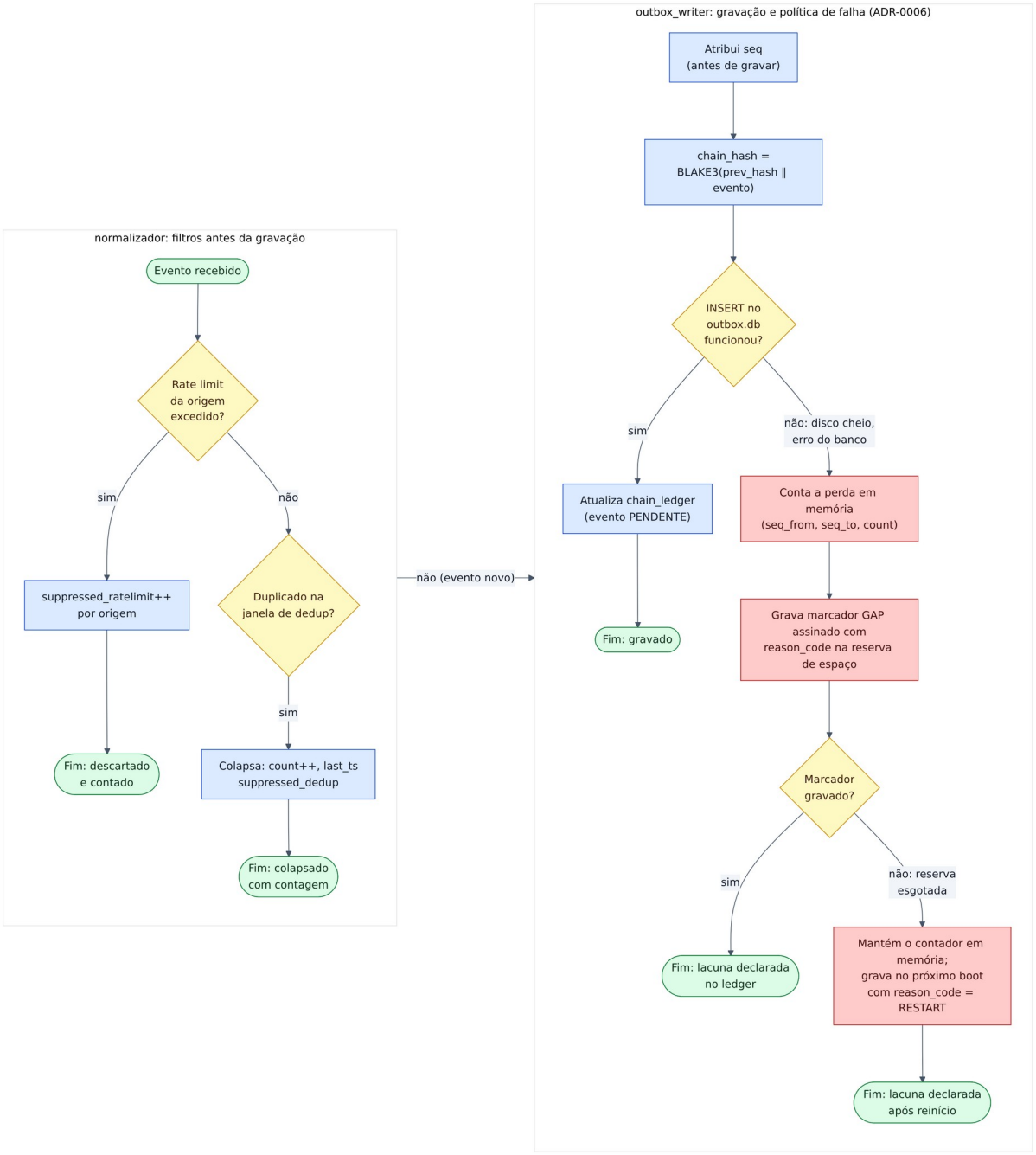


Figura 17: Fluxo de decisão do ADR-0006: filtros, gravação, GAP e reserva de espaço.

8.6 Implantação e ativação

O primeiro diagrama mostra quais artefatos (binários, agent.toml, certificados, bancos SQLite) ficam em cada nó; o segundo descreve o script idempotente de ativação, que pode ser reexecutado com segurança e falha com diagnóstico explícito, sem alterar a rede nem a gerência.

Figura 18 — Implantação: serviços, artefatos e configuração (v1.3)

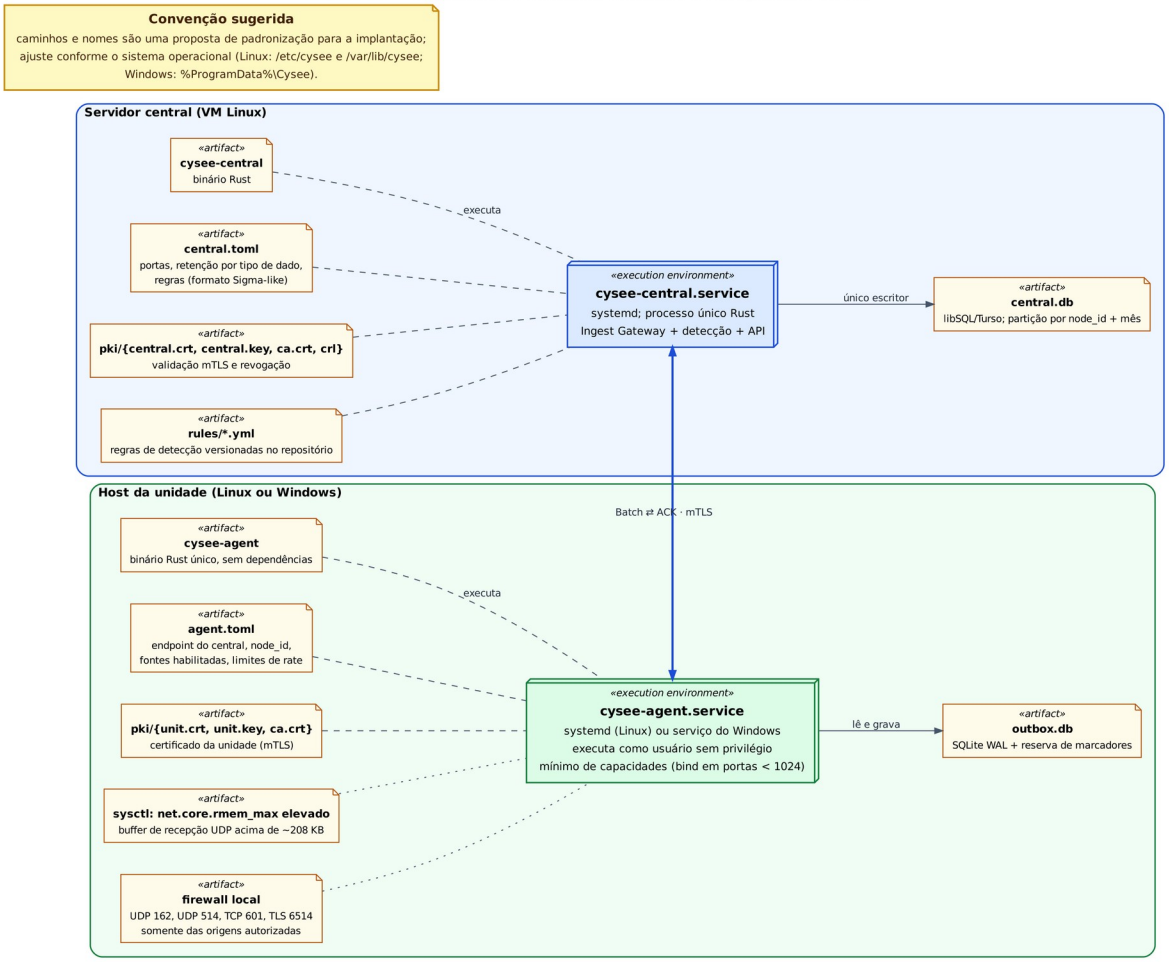


Figura 18: Implantação dos artefatos e arquivos de configuração por nó.

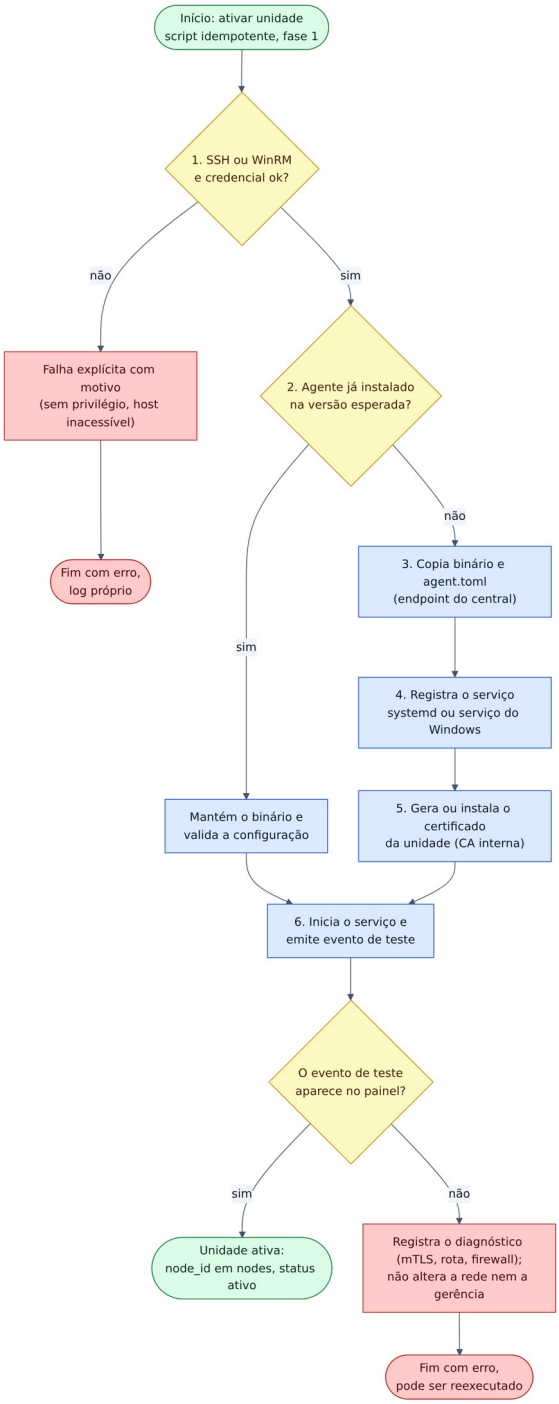


Figura 19: Atividade do script de ativação de uma unidade (fase 1).